

มหาวิทยาลัยราชภัฏบุรีรัมย์ — Strategic Performance Tracking System

เอกสารเตรียม

สอบ

คู่มือเตรียมสอบโครงการจบ: การทำงานระบบ การแก้ไขโค้ด และแนวทางการตอบข้อซักถาม
(Defense Masterplan)

ฉบับเข้าใจง่าย &
ครบถ้วน

คู่มือเตรียมสอบโครงการจบ: ระบบติดตามและประเมินผลโครงการตามยุทธศาสตร์มหาวิทยาลัย

BRU Strategic Tracking System — Defense & Presentation Masterplan (ฉบับเข้าใจง่าย)

สารบัญหัวข้อหลัก

- การเริ่มทดสอบการทำงานระบบ: ลำดับการทดสอบตาม Role พร้อมฉากการนำเสนอ (Testing Workflow & Presentation Script)
- วิธีรับมือเมื่อถูกสั่งให้แก้ไขโค้ดสด (Live Coding & Customization Defense)
 - 2.1 การเปลี่ยนสีระบบ (Color Palette & Theme)
 - 2.2 การแก้ไขข้อความ (Text, Labels & Alert Messages)
 - 2.3 การแก้ไขโค้ดตรรกะระบบ (Business Logic & Validation)
- เครื่องมือที่ใช้ในการพัฒนาระบบ (Tech Stack & Development Tools)
- โครงสร้างไฟล์ทั้งหมด หน้าที่ของแต่ละไฟล์ และการทำงานร่วมกัน (Full System Architecture & File Breakdown)
 - 4.1 แผนผังโฟลเดอร์และไฟล์ทั้งหมดในระบบ (Inventory Tree: Database, Backend, Frontend)
 - 4.2 คำอธิบายหน้าที่ของแต่ละไฟล์ในระบบอย่างละเอียด (File-by-File Detailed Explanation)
 - 4.3 โครงสร้างหน้าจอหลักและองค์ประกอบ UI (Master Layout: Topbar, Sidebar, Main Canvas)
 - 4.4 แผนผังและกลไกการทำงานร่วมกันตั้งแต่หน้าบ้านถึงฐานข้อมูล (End-to-End Collaboration & Sequence Diagram)
- การ Deploy ระบบ: เครื่องมือ วิธีการทำงาน และการตั้งค่า (Deployment Architecture)
 - 5.1 การ Deploy ฝั่ง Frontend ด้วย Vercel
 - 5.2 การ Deploy ฝั่ง Backend ด้วย Render / Railway หรือ VPS
 - 5.3 การเชื่อมต่อฐานข้อมูล (Database Deployment)
- ชื่อตารางฐานข้อมูล 14 ตารางและความหมาย (Database Architecture & Data Dictionary)
- คำถามที่คาดว่าจะถามเกี่ยวกับระบบ พร้อมแนวทางการตอบ (Top 12 Defense Q&A)

1. การเริ่มทดสอบการทำงานระบบ: ลำดับการทดสอบตาม Role

หลักการง่ายๆ: นำเสนอแบบ "ต้นน้ำ → กลางน้ำ → ปลายน้ำ → หลังบ้าน"

เริ่มจาก อาจารย์ (คนป้อนข้อมูล) → คณบดี (คนกำกับคณะ) → อธิการบดี (คนสั่งการมหาวิทยาลัย) → แอดมิน (คนคุมระบบ)

ลำดับการเดินเรื่องนำเสนอระบบ

1. อาจารย์ (TEACHER) : สร้างโครงการ → ล็อกแผน → บันทึกรูป → คำนวณ % สะสม
2. คณบดี (DEAN) : ตรวจสอบบอร์ดเฉพาะคณะ → ดูผลงานสาขา → สั่งการ Dean
3. อธิการบดี (PRESIDENT) : ตรวจสอบรวม มรภ. → ตรวจสอบโครงการติดธงแดง → สั่งการ Pres
4. แอดมิน (ADMIN) : จัดการยุทธศาสตร์ → ดูแลบัญชีผู้ใช้ → จัดการปัญหา

ตารางบัญชีผู้ใช้งานสำหรับทดสอบ (Demo Test Accounts)

บทบาท (Role)	บัญชีผู้ใช้ (Username)	รหัสผ่าน (Password)	ใครเป็นผู้ใช้งาน?
TEACHER	teacher	123456	อ.สมชาย ใจดี (สาขาวิทยาการคอมพิวเตอร์ คณะวิทยาศาสตร์)
DEAN	dean	123456	ผศ.ดร.คณบดีคณะวิทยาศาสตร์
PRESIDENT	president	123456	รศ.ดร.อธิการบดีมหาวิทยาลัยราชภัฏบุรีรัมย์
ADMIN	admin	admin1234	ผู้ดูแลระบบสารสนเทศส่วนกลาง

รายละเอียดขั้นตอนการสาริตและบทพูดทีละฉาก (Scene-by-Scene Screenplay)

จุดเริ่มต้น (1 นาทีแรก): เกริ่นเปิดตัวปัญหาและวัตถุประสงค์

- บทพูดแนะนำตัว:

"สวัสดีครับอาจารย์ทุกท่าน ระบบนี้คือ 'ระบบติดตามและประเมินผลโครงการตามยุทธศาสตร์ มหาวิทยาลัยราชภัฏบุรีรัมย์' ครับ"

ปัญหาเดิมคือมหาวิทยาลัยติดตามโครงการผ่านเอกสารกระดาษและ Excel ทำให้ข้อมูลไม่เป็นปัจจุบัน ข้อมูลกระจัดกระจาย และผู้บริหารไม่รู้ว่าโครงการไหนกำลังติดขัด

ระบบนี้จึงถูกสร้างขึ้นเพื่อแปลงยุทธศาสตร์ลงสู่การปฏิบัติจริงที่มีหลักฐานเชิงประจักษ์ (Evidence-based) และช่วยให้ผู้บริหารติดตามเฉพาะโครงการที่มีปัญหาได้ทันท่วงที (Management by Exception) ครับ"

สแต็ปที่ 1: เข้าใช้งานในมุมมองอาจารย์ผู้รับผิดชอบโครงการ (Role: TEACHER)

• ล็อกอิน: /

• สิ่งที่ต้องกดโชว์:

1. เข้าหน้าแรกจะเห็น "แดชบอร์ดงานของฉัน" (สรุปเฉพาะโครงการที่ตนเองดูแล)
2. กดเมนูด้านซ้าย "เพิ่มโครงการใหม่" ()
3. โชว์จุดเด่น 1 (Cascading Dropdown ยุทธศาสตร์ 4 ระดับ):
 - เลือก ประเด็นการพัฒนาท้องถิ่น (LDI) → Dropdown แผนงานหลัก (Strategy) จะกรองเฉพาะตัวเลือกที่เกี่ยวข้องขึ้นมาอัตโนมัติ
 - เลือก แผนงานย่อย (Sub-Strategy) → แล้วเลือก โครงการหลัก (Main Project - MP)
 - ชี้ให้กรรมการดู: แดปสีม่วงอ่อน "Strategic Alignment Pipeline" ด้านล่าง จะแสดงเส้นทางความเชื่อมโยงยุทธศาสตร์แบบ 4 ชั้นยืนยันความสอดคล้อง
4. โชว์จุดเด่น 2 (การผูกผู้ร่วมรับผิดชอบ): ตีเลือกอาจารย์ท่านอื่นในคณะเดียวกันมาร่วมดูแลโครงการ
5. กรอกงบประมาณ เช่น บาท, เป้าหมาย , หน่วยนับ , กำหนดวันเริ่ม-สิ้นสุด → กดปุ่ม "บันทึกโครงการ"
6. ระบบพาเข้าหน้า "รายละเอียดโครงการ" () → กดปุ่ม "เพิ่มกิจกรรม"
 - ใส่ค่าใช้จ่ายจริง บาท, ผลผลิตที่ทำได้ คน และแนบรูปภาพลงพื้นที่จริง → กดบันทึก
 - ชี้ให้กรรมการดู: หลอด Progress Bar ขยับจำนวนใหม่เป็น 50.00% ทันทีแบบ Real-time
7. โชว์จุดเด่น 3 (ระบบล็อกแผนงาน - Plan Locking):
 - พอกิจกรรมถูกบันทึกแล้ว กดปุ่ม "แก้ไขโครงการ"
 - ชี้ให้กรรมการดู: ช่องงบประมาณและเป้าหมายจะขึ้นแม่กุญแจ  สีเทาพิมพ์แก้ไขไม่ได้แล้ว เพื่อป้องกันการลดเป้าหมายย้อนหลังตามหลักธรรมาภิบาล

สแต็ปที่ 2: เข้าใช้งานในมุมมองคณบดี (Role: DEAN)

• ล็อกอิน: /

• สิ่งที่ต้องกดโชว์:

1. เข้าหน้าแรกจะเห็น "แดชบอร์ดภาพรวมคณะวิทยาศาสตร์"
2. ชี้ให้กรรมการดู Data Isolation (การจำกัดสิทธิ์ระดับคณะ):
 - ตัวเลขงบประมาณ โครงการ และกราฟทั้งหมดจะแสดงเฉพาะคณะวิทยาศาสตร์เท่านั้น ไม่เห็นข้อมูลของคณะอื่น
3. โชว์ตาราง Cross-Department Matrix เปรียบเทียบผลงาน อัตราเบิกจ่ายงบประมาณระหว่างสาขาวิชาในคณะ
4. โชว์การสั่งการระดับคณะ (Dean Directive):

- คลิกเปิดโครงการในคณะ → พิมพ์ข้อสั่งการเร่งรัดงาน เช่น "ขอให้สาขาเร่งเบิกจ่ายงวดแรกภายในสัปดาห์นี้" → กดบันทึก
- อธิบายว่าระบบมี **IDOR Protection** คนบดจะไม่สามารถส่งคำสั่งข้ามไปยังโครงการของคณะอื่นได้เด็ดขาด

สแต็ปที่ 3: เข้าใช้งานในมุมมองอธิการบดี/ผู้บริหารระดับสูง (Role: PRESIDENT)

- ล็อกอิน: `president` / `123456`
- สิ่งที่ต้องกดโชว์:
 1. เข้าหน้าแรกจะเห็น **"Strategic Dashboard ภาพรวมมหาวิทยาลัย"**
 2. โชว์ภาพรวมงบประมาณแผ่นดินรวม อัตราการเบิกจ่าย และสัดส่วนความก้าวหน้าทุกยุทธศาสตร์
 3. โชว์จุดเด่น **Management by Exception (โครงการติดธงแดง - Red Flags):**
 - ชี้ให้ดูกล่องสีแดง **"โครงการที่ต้องเร่งรัดติดตาม (Critical Bottlenecks)"**
 - อธิบายว่าระบบคัดกรองโครงการที่หมดเวลาแล้วแต่ยังไม่เสร็จ หรือเวลาผ่านไปเกิน 50% แต่ยังไม่ถึง 25% ขึ้นมาเตือนผู้บริหารอัตโนมัติ
 4. โชว์การสั่งการอธิการบดี (President Directive):
 - คลิกเปิดโครงการติดธงแดง → พิมพ์ข้อสั่งการระดับนโยบาย ส่งตรงไปยังหน้าจอของอาจารย์ผู้รับผิดชอบทันที

สแต็ปที่ 4: เข้าใช้งานในมุมมองผู้ดูแลระบบ (Role: ADMIN)

- ล็อกอิน: `admin` / `admin1234`
- สิ่งที่ต้องกดโชว์:
 1. คลิกเมนู **"จัดการข้อมูลระบบ (Master Data)"** (`/master-data`)
 2. แสดงแท็บจัดการยุทธศาสตร์ 4 ระดับ, ปีงบประมาณ (เปิด/ปิด active), แหล่งงบประมาณ, คณะ/สาขา และจัดการบัญชีผู้ใช้งาน
 3. แสดงหน้า **"รายงานปัญหา (Issue Reports)"** ที่รับเรื่องแจ้งซ่อม/ข้อขัดข้องจากผู้ใช้ทุกระดับ เพื่อให้ Admin ตรวจสอบและอัปเดตสถานะ

2. วิธีรับมือเมื่อถูกสั่งให้แก้โค้ดสด (Live Coding Defense)

เทคนิคก่อนเริ่มพิมพ์:

1. ทวนโจทย์ให้ชัดเจน
2. บอกชื่อไฟล์ที่จะเปิดก่อนแตะแป้นพิมพ์
3. บอกจุดที่จะแก้สั้นๆ แล้วลงมือแก้ → บันทึกลง → แสดงผลทันที

2.1 การเปลี่ยนสีระบบ (Color Palette & Theme)

โจทย์ ก: "ลองเปลี่ยนสีหลัก (Primary Theme) ของทั้งระบบเป็นสีน้ำเงิน หรือสีเขียวให้ดูหน่อย"

- ไฟล์ที่ต้องเปิด: `frontend/tailwind.config.js` (บรรทัดที่ 10–14)
- วิธีแก้: เปลี่ยนรหัสสี HEX ของ `primary.DEFAULT` เพียงบรรทัดเดียว:

```
// เปลี่ยนจากสีม่วงเดิม (#6C3BFF) เป็น:
primary: {
  DEFAULT: '#2563EB', // สีน้ำเงิน (หรือ #059669 สีเขียวมรกต)
  dark: '#1D4ED8',
  light: '#DBEAFF',
}
```

- คำอธิบายต่อกรรมการ: "ระบบเราใช้ Semantic Tokens ของ Tailwind CSS ครับ การกำหนดสีหลักไว้ที่ `primary.DEFAULT` เพียงจุดเดียว ทำให้ปุ่ม ไอคอน และแถบไฮไลต์ทั้งระบบเปลี่ยนธีมสีตามทันทีครับ"

โจทย์ ข: "เปลี่ยนสีพื้นหลังหรือสีขอบของ Sidebar เมนูด้านซ้าย"

- ไฟล์ที่ต้องเปิด: `frontend/src/components/Sidebar.jsx`
- วิธีแก้: ค้นหาแท็ก `<aside>` แล้วเปลี่ยนคลาสจาก `bg-white` เป็น `bg-slate-900 text-white` (เปลี่ยนเป็น โทนมืด)

2.2 การแก้ไขข้อความ (Text, Labels & Alert Messages)

โจทย์ ก: "เปลี่ยนหัวข้อหน้าเว็บ หรือชื่อระบบตรง Topbar"

- ไฟล์ที่ต้องเปิด: `frontend/src/components/Topbar.jsx` (บรรทัด 24–60 ในฟังก์ชัน `getPageTitle`)
- วิธีแก้: แก้ไขข้อความในคำสั่ง `return ' ... '` เช่น จาก `'แดชบอร์ดงานของฉัน'` เป็น `'ระบบติดตามยุทธศาสตร์ มรภ.บุรีรัมย์'`

โจทย์ ข: "เปลี่ยนข้อความแจ้งเตือน Pop-up (SweetAlert2) เมื่อบันทึกสำเร็จ"

- ไฟล์ที่ต้องเปิด: `frontend/src/pages/teacher/ProjectForm.jsx`
- วิธีแก้: ค้นหา `Swal.fire` แล้วเปลี่ยนข้อความใน `title:` หรือ `text:`

```
Swal.fire({
  icon: 'success',
  title: 'บันทึกข้อมูลเรียบร้อยแล้ว!',
  text: 'ข้อมูลโครงการถูกส่งเข้าสู่ระบบยุทธศาสตร์แล้ว'
});
```

2.3 การแก้ไขโค้ดตรรกะระบบ (Business Logic & Validation)

โจทย์ ก: "ลองแก้ไขให้ผู้ใช้งานกรอกเป้าหมายน้อยกว่า 10"

- หน้าบ้าน (Frontend): เปิด `frontend/src/pages/teacher/ProjectForm.jsx` → ค้นหา `targetCount` แก้เป็น `min: { value: 10, message: 'เป้าหมายต้องไม่ต่ำกว่า 10' }`
- หลังบ้าน (Backend): เปิด `backend/middleware/validation.middleware.js` → แก้เป็น `.isInt({ min: 10 })`
- ไช้กรรมการ: ทดลองกรอกเลข 5 แล้วกดบันทึก ระบบจะแจ้งเตือนทั้งหน้าบ้านและถูกบล็อกที่หลังบ้าน

โจทย์ ข: "ซ่อนปุ่มลบโครงการ ไม่ให้อาจารย์เห็น ให้เห็นเฉพาะ ADMIN"

- ไฟล์ที่ต้องเปิด: `frontend/src/pages/teacher/ProjectDetails.jsx`
- วิธีแก้: นำตัวแปร `user` มาครอบเงื่อนไข:

```
{user?.role === 'ADMIN' && (
  <button onClick={handleDelete} className="btn-danger">ลบโครงการ</button>
)}
```

โจทย์ ค: "ถ้ากรรมการสั่งให้แก้โครงสร้างตารางฐานข้อมูลสด"

- คำตอบแบบมืออาชีพ:

"ในระบบจริงเราใช้ Prisma ORM การแก้ตารางจะต้องเริ่มจากการแก้ไฟล์ `schema.prisma` → รันคำสั่ง `npx prisma migrate dev` เพื่อให้ MySQL สร้างคอลัมน์ใหม่ แล้วจึงไปอัปเดต Controller แต่เนื่องจากการ Migrate สดตอนนี้อาจกระทบกับชุดข้อมูลทดสอบเดิม ผมขออนุญาต เขียนโค้ดจำลองในไฟล์ และอธิบายขั้นตอนการทำงานที่ละเอียด ให้อาจารย์เห็นภาพแทนได้ไหมครับ"

3. เครื่องมือที่ใช้ในการพัฒนาระบบ (Tech Stack)

ส่วนของระบบ	เครื่องมือที่ใช้	บทบาทและหน้าที่ (ทำไมต้องเลือกตัวนี้?)
Frontend UI	React 19 + Vite	React จัดการหน้าจอด้วย Virtual DOM ทำงานแบบ SPA โหลดหน้าเร็ว ส่วน Vite ช่วยให้ Build และ Hot-Reload ได้ทันทีใน 3 วินาที
Styling	Tailwind CSS	Utility-first CSS กำหนดขนาด สเปซ และสีเส้นได้แม่นยำ รองรับ Responsive ทุกขนาดหน้าจอ
Icons	React-Icons (Feather)	ไอคอน Vector SVG คมชัด ไม่แตก ปรับสีและขนาดผ่านคลาส Tailwind ได้ง่าย
Form Management	React Hook Form	จัดการฟอร์มขนาดใหญ่ที่มี Dropdown ซับซ้อนได้ลื่นไหล ไม่เกิดการ Re-render ทั้งหน้าแบบพรวดเดียว
Data Visualization	Chart.js + React-Chartjs-2	สร้างกราฟสรุปสถิติงบประมาณและผลงาน เรนเดอร์บน HTML5 Canvas ลื่นไหลสวยงาม
Alerts & Modals	SweetAlert2	แจ้งเตือน Pop-up และ Dialog ยืนยันการลบข้อมูล สวยงาม มีมิติ แทนที่ <code>alert()</code> ธรรมดา
Backend API	Node.js + Express.js	Node.js ทำงานแบบ Non-blocking I/O รองรับ Request ได้พร้อมกันจำนวนมาก ส่วน Express มี Middleware ที่แข็งแกร่ง
Database & ORM	MySQL 8.0 + Prisma ORM	MySQL เก็บข้อมูลสัมพันธ์มั่นคงสูง ส่วน Prisma มี Type-safety และ Parameterized Queries ป้องกัน SQL Injection ได้ 100%
Security & Auth	JWT + Bcrypt.js	Bcrypt แฮชรหัสผ่าน 10 รอบ และใช้ JWT สำหรับ Stateless Authentication ควบคุมการเข้าถึงด้วย RBAC
File Uploading	Multer	จัดการรับไฟล์รูปภาพหลักฐานกิจกรรม ตรวจสอบนามสกุลไฟล์ และจัดเก็บลงโฟลเดอร์ <code>/uploads</code> อย่างเป็นระบบ

4. โครงสร้างไฟล์ทั้งหมด หน้าที่ของแต่ละไฟล์ และการทำงานร่วมกัน

4.1 แผนผังไฟล์เดอร์และไฟล์ทั้งหมดในระบบ (Inventory Tree)

c:\St_bru\	
database/	(สคริปต์ DDL, Data Dictionary และ ERD)
DataDictionary.md	– เอกสารพจนานุกรมข้อมูล อธิบายโครงสร้างฟิลด์และชนิดข้อมูลของทั้ง 14 ตาราง
ERD.md	– เอกสารแผนภาพความสัมพันธ์เอนทิตี (ER Diagram) ของฐานข้อมูล
schema.sql	– สคริปต์ DDL ภาษา SQL ตั้งเดิมสำหรับสร้าง 14 ตารางบน MySQL
seed.sql	– สคริปต์ INSERT INTO ภาษา SQL สำหรับใส่ข้อมูลเริ่มต้นของระบบ
postman/	(ชุดทดสอบ API สำหรับโปรแกรม Postman)
BRU_Strategic_Tracking_API_Test_Collection.postman_collection.json	– รวมชุดทดสอบ API ทั้งหมด 8 หมวดหมู่ (25+ Requests) พร้อม Script เก็บ Token
BRU_Strategic_Tracking_Environment.postman_environment.json	– ตัวแปร Environment ทดสอบระบบ (base_url, token, บัญชี 4 Role)
README.md	– คู่มือการ Import และ ทดสอบ API ผ่าน Postman ใน 2 คลิก
backend/	(ระบบเซิร์ฟเวอร์ Node.js + Express + Prisma ORM)
.env	– ตัวแปรสภาพแวดล้อมระบบ (DATABASE_URL, JWT_SECRET, PORT)
app.js	– จุดเริ่มต้นเซิร์ฟเวอร์ Express, รวม Middleware และเชื่อมต่อ Route
หลัก	
package.json / package-lock.json	– กำหนด Dependencies และคำสั่งสคริปต์รันของ Backend
seed-projects.js	– สคริปต์สร้างข้อมูลโครงการและกิจกรรมจำลองสำหรับทดสอบ
verify-endpoints.js	– สคริปต์ทดสอบยิง API ตรวจสอบสถานะการทำงานของ Endpoint ต่างๆ (การตั้งค่าระบบและการเชื่อมต่อภายนอก)
config/	
prisma.js	– Singleton Instance ของ PrismaClient ป้องกัน Connection ซ้ำ
ข้อ	
cloudinary.js	– ตั้งค่าเชื่อมต่องานจัดเก็บรูปภาพ Cloudinary (ตัวเลือกเสริม)
middleware/	
auth.middleware.js	– (ตัวคัดกรองคำขอ Request ก่อนเข้า Controller) ตรวจสอบ JWT Token (authenticate) และตรวจสอบสิทธิ์ตาม Role (authorize)
validation.middleware.js	– ตรวจสอบความถูกต้องของข้อมูล Input ป้องกันข้อมูลผิดรูปแบบ
upload.middleware.js	– ตัวจัดการ Multer ตรวจสอบนามสกุลไฟล์ภาพและจำกัดขนาดไม่เกิน 5MB (กำหนดเส้นทาง Endpoint และจับคู่กับ Controller)
routes/	
auth.routes.js	– เส้นทาง API ล็อกอิน, ตรวจสอบผู้ใช้ปัจจุบัน, และเปลี่ยนรหัสผ่าน
project.routes.js	– เส้นทาง API เพิ่ม ลบ แก้ไข ล็อกแผนงาน และค้นหาโครงการ
activity.routes.js	– เส้นทาง API เพิ่มกิจกรรมย่อย อัปโหลดรูปภาพ และคำนวณผลผลิต
dashboard.routes.js	– เส้นทาง API ดึงข้อมูลสรุปแดชบอร์ดตามบทบาทผู้ใช้งาน (Role)
directive.routes.js	– เส้นทาง API ส่งข้อสั่งการเร่งรัดงานของผู้บริหาร (Dean / President)
master.routes.js	– เส้นทาง API จัดการข้อมูลพื้นฐานระบบ 9 หมวด (ยุทธศาสตร์, คณะ,
สาขา)	
issue.routes.js	– เส้นทาง API ส่งเรื่องแจ้งปัญหาการใช้งานและอัปเดตสถานะปัญหา
report.routes.js	– เส้นทาง API รวบรวมข้อมูลสรุปโครงการเพื่อจัดทำรายงาน (ตรวจทางธุรกิจ Business Logic และประมวลผลข้อมูล)
controllers/	
auth.controller.js	– ตรวจสอบรหัสผ่าน (Bcrypt), สร้าง JWT Token และจัดการสิทธิ์ผู้ใช้
project.controller.js	– คำนวณความก้าวหน้าโครงการ ระบบล็อกแผนงาน และตรวจสอบสิทธิ์ IDOR
activity.controller.js	– บันทึกผลการดำเนินกิจกรรม คำนวณยอดเงินสะสม และจัดการรูปภาพหลักฐาน
dashboard.controller.js	– ประมวลผลสถิติ กรองโครงการติดธงแดง (Red Flags) ตามข้อยกเว้น MBE
master.controller.js	– ตรวจจับการเพิ่ม/ลบ/แก้ไขข้อมูลยุทธศาสตร์ 4 ระดับ และหน่วยงาน
issue.controller.js	– รับเรื่องแจ้งปัญหา บันทึกรายละเอียด และเปลี่ยนสถานะโดยผู้ดูแลระบบ
report.controller.js	– คำนวณยอดรวมงบประมาณและผลผลิตตามยุทธศาสตร์เพื่อออกรายงาน (การจัดการฐานข้อมูลด้วย Prisma ORM)
prisma/	
schema.prisma	– แบบจำลองฐานข้อมูล 14 ตาราง Enums ความสัมพันธ์ และกฎความปลอดภัย
seed.js	– สคริปต์หลักสำหรับสร้าง Master Data และบัญชีทดสอบครบทุก Role

	- seed-strategic-plan.js - seed-admin-only.js - seed-production-masterdata.js - setup-4tiers.js - migrations/ - uploads/	– สคริปต์นำเข้าโครงสร้างยุทธศาสตร์ 4 ระดับ (LDI, Plan, Sub, MP) – สคริปต์สร้างบัญชีผู้ดูแลระบบ (Admin) โดยเฉพาะ – สคริปต์ตั้งค่า Master Data สำหรับใช้งานบนสภาพแวดล้อมจริง – สคริปต์จัดหมวดยุทธศาสตร์ 4 ระดับให้ตรงตามแผนของมหาวิทยาลัย – ไฟล์เตอร์เก็บประวัติและไฟล์สคริปต์การอัปเดตโครงสร้างฐานข้อมูล (ไฟล์เตอร์จัดเก็บไฟล์รูปภาพหลักฐานกิจกรรมที่อัปโหลดขึ้นเซิร์ฟเวอร์)
	- frontend/ - index.html - vite.config.js - tailwind.config.js	(ระบบหน้าบ้าน React 19 + Vite + Tailwind CSS) – โครงสร้าง HTML หลัก จัดติดตั้ง Root Element ของ React – กำหนดค่า Vite, ตั้งค่า Proxy เชื่อมต่อไปยัง Backend และพอร์ต – กำหนดสีหลัก (primary: #6C3BFF) ฟอนต์ และ Utility คลาสของ
ระบบ	postcss.config.js	– ตั้งค่า PostCSS สำหรับคอมไพล์สไคล์ร่วมกับ Tailwind และ
Autoprefixer	vercel.json	– กฎ Rewrite ส่งทุก URL ไปที่ index.html ป้องกันหน้า 404 บน
Vercel	- package.json / package-lock.json - src/ - main.jsx - App.jsx - index.css - layouts/ - AppLayout.jsx	– กำหนด Dependencies และค่าสคริปต์รันของ Frontend (โค้ดต้นฉบับฝั่งหน้าบ้าน) – จุดเริ่มต้น React นำ App ไปเรนเดอร์ใน DOM และห่อหุ้มด้วย Router – จัดการเส้นทาง URL (Routing) และระบบป้องกันสิทธิ์ (Role Guard) – ไฟล์สไคล์ CSS หลัก และการตั้งค่าพื้นฐานของ Tailwind CSS (โครงสร้างหน้าจอหลัก) – Master Layout รวม Sidebar + Topbar + Outlet (พื้นที่แสดง
เนื้อหา)	- components/ - Topbar.jsx - Sidebar.jsx - CustomSelect.jsx - ExecutiveProjectModal.jsx - ProfileModal.jsx - ReportIssueModal.jsx - ErrorBoundary.jsx - contexts/ - AuthContext.jsx - services/ - api.js - utils/ - imageUrl.js - statusHelper.js - pages/ - auth/ - Login.jsx - teacher/ - Dashboard.jsx - Projects.jsx - ProjectForm.jsx	(ชิ้นส่วน UI ที่นำมาใช้ซ้ำได้) – แถบเมนูด้านบน แสดงข้อมูลระบบ แจ้งเตือน และโปรไฟล์ผู้ใช้ – เมนูแถบข้างด้านซ้าย เปลี่ยนรายการตาม Role ของผู้ใช้โดยอัตโนมัติ – Dropdown พิเศษที่รองรับการค้นหาคำเลือก (Searchable Dropdown) – หน้าต่าง Pop-up สรุปข้อมูลโครงการสำหรับผู้บริหาร – หน้าต่าง Pop-up แสดงข้อมูลผู้ใช้และฟอร์มเปลี่ยนรหัสผ่าน – หน้าต่าง Pop-up สำหรับกรอกและส่งเรื่องแจ้งปัญหาไปยังแอดมิน – ตัวดักจับข้อผิดพลาดของ UI เพื่อป้องกันไม่ให้หน้าเว็บล่มทั้งระบบ (ตัวจัดการ State ส่วนกลางของระบบ) – จัดการ State การล็อกอิน ข้อมูลผู้ใช้ สิทธิ์ และฟังก์ชัน Login/Logout (การเชื่อมต่อ API หลังบ้าน) – กำหนดค่า Axios Client และดักแนบ JWT Token ในทุกคำขออัตโนมัติ (ฟังก์ชันช่วยเหลือทั่วไป) – แปลง Path สัมพัทธ์ของรูปภาพให้เป็น Full URL สำหรับแสดงผลบนหน้าเว็บ – ฟังก์ชันแปลงรหัสสถานะโครงการ/กิจกรรมเป็นสี Badge และข้อความภาษาไทย (หน้าจอแสดงผลตามบทบาทผู้ใช้) (หน้าจอสำหรับการยืนยันตัวตน) – หน้าจอกรอกชื่อผู้ใช้และรหัสผ่านเพื่อเข้าสู่ระบบ (หน้าจอสำหรับอาจารย์ผู้รับผิดชอบโครงการ) – แดชบอร์ดสรุปภาพรวมโครงการเฉพาะที่ตนเองรับผิดชอบ – หน้ารายการโครงการทั้งหมดของตนเอง พร้อมปุ่มสร้างโครงการใหม่ – ฟอร์มเพิ่ม/แก้ไขโครงการ พร้อม Cascading Dropdown ยุทธศาสตร์ 4
ระดับ	- ProjectDetails.jsx - ActivitiesList.jsx - Gallery.jsx - dean/ - Dashboard.jsx - Projects.jsx - Reports.jsx - president/ - Dashboard.jsx - Projects.jsx - Reports.jsx - admin/ - Dashboard.jsx - MasterData.jsx	– หน้ารายละเอียดโครงการ บันทึกกิจกรรม และแสดงสถานะล็อกแผนงาน – หน้ารายการกิจกรรมย่อยทั้งหมดของโครงการ – หน้าแสดงอัลบั้มรูปภาพหลักฐานการลงพื้นที่จริงทั้งหมด (หน้าจอสำหรับคณบดี) – แดชบอร์ดสรุปผลงานระดับคณะ และปุ่มออกข้อสั่งการเร่งรัดงาน – หน้ารายการโครงการเฉพาะภายในคณะของตนเอง – หน้ารายงานสรุปผลงาน เปรียบเทียบผลการดำเนินงานระหว่างสาขาวิชา (หน้าจอสำหรับอธิการบดี/ผู้บริหารระดับสูง) – แดชบอร์ดรวมมหาวิทยาลัย รายการรณรงค์ และข้อสั่งการระดับนโยบาย – หน้ารายการโครงการทั้งหมดทุกคณะในมหาวิทยาลัย – หน้ารายงานสถิติงบประมาณและการบรรลุเป้าหมายภาพรวมมหาวิทยาลัย (หน้าจอสำหรับผู้ดูแลระบบ) – แดชบอร์ดสถิติผู้ใช้งาน โครงการ และสถานะปัญหาในระบบ – หน้าจัดการข้อมูลระบบ 9 หมวด (ยุทธศาสตร์, คณะ, สาขา, แหล่งเงินทุน)

Projects.jsx	– หน้าบริหารจัดการโครงการทุกโครงการในระบบ
AdminActivities.jsx	– หน้าตรวจสอบและบริหารจัดการกิจกรรมย่อยของทุกโครงการ
Issues.jsx	– หน้าระบบจัดการเรื่องร้องเรียน/แจ้งปัญหาการใช้งาน
Reports.jsx	– หน้ารวมรายงานเชิงสถิติขั้นสูงสำหรับผู้ดูแลระบบ
executive/	(หน้าจอรายละเอียดสำหรับระดับผู้บริหาร)
ProjectDetail.jsx	– หน้าเจาะลึกโครงการสำหรับผู้บริหาร ดูไหมไลน์และประวัติการสั่งการ

4.2 คำอธิบายหน้าที่ของแต่ละไฟล์ในระบบอย่างละเอียด

1) หมวดฐานข้อมูล (`database/`)

- **schema.sql** : คำสั่ง SQL ดั้งเดิม (DDL) สำหรับสร้าง 14 ตาราง, กำหนด Primary Keys, Foreign Keys และ Constraints บน MySQL
- **seed.sql** : ชุดคำสั่ง `INSERT INTO` ข้อมูลจำลองและยุทธศาสตร์ตั้งต้น
- **DataDictionary.md** : พจนานุกรมข้อมูลฉบับสมบูรณ์ อธิบายชื่อฟิลด์ ชนิดข้อมูล และหน้าที่ของทุกคอลัมน์ใน 14 ตาราง
- **ERD.md** : แผนผังความสัมพันธ์เอนทิตี (Entity-Relationship Diagram) แสดงเส้นโยง 1:N และ Many-to-Many

2) หมวดหลังบ้าน (`backend/`)

- **app.js** : จุดเริ่มต้นของเซิร์ฟเวอร์ Express รวม Middleware ทั้งหมด (CORS, Rate Limiter, Morgan, JSON Parser), แมป Route ปลายทาง และให้บริการไฟล์เดอร์ภาพ `/uploads`
- **config/prisma.js** : ตัวสร้าง Singleton Instance ของ PrismaClient ป้องกันการเปิด Connection ซ้ำซ้อน
- **.env** : จัดเก็บตัวแปรระบบที่เป็นความลับ เช่น `DATABASE_URL`, `JWT_SECRET`, `PORT`
- **prisma/schema.prisma** : แม่แบบฐานข้อมูลของระบบ กำหนดโมเดล 14 ตาราง, Enums สิทธิ์, ความสัมพันธ์ Cascade และฟิลด์ข้อสั่งการผู้บริหาร
- **prisma/seed.js** : สคริปต์สร้างข้อมูลตั้งต้น (User แอดมิน, ผู้บริหาร, อาจารย์, ยุทธศาสตร์ 4 ระดับ, คณะ/สาขา)

มิดเดิลแวร์ความปลอดภัย (`middleware/`):

- **auth.middleware.js** : มี `authenticate` (ตรวจ JWT Bearer Token) และ `authorize` (ตรวจสอบสิทธิ์ Role)
- **validation.middleware.js** : ตรวจสอบความถูกต้องของข้อมูลอินพุต เช่น รูปแบบวันที่, ตัวเลขงบประมาณ ไม่ติดลบ
- **upload.middleware.js** : ใช้ Multer จัดการไฟล์รูปภาพ ตรวจสอบนามสกุลและจำกัดขนาดไม่เกิน 5MB

เส้นทาง API (`routes/`):

- **auth.routes.js** : เส้นทางล็อกอิน, ดึงโปรไฟล์ และเปลี่ยนรหัสผ่าน
- **project.routes.js** : เส้นทางเพิ่ม/แก้ไข/ลบ/ล็อก โครงการ
- **directive.routes.js** : เส้นทางส่งข้อสั่งการระดับคณบดีและอธิการบดี (`/dean`, `/president`)

- `activity.routes.js` : เส้นทางบันทึกกิจกรรมย่อย และอัปโหลด/ลบรูปภาพหลักฐาน
- `dashboard.routes.js` : เส้นทางดึงข้อมูลสรุปแดชบอร์ดตาม Role
- `master.routes.js` : เส้นทางจัดการข้อมูลหลัก 9 หมวดหมู่
- `issue.routes.js` : เส้นทางรับแจ้งปัญหาและอัปเดตสถานะการแก้ไข
- `report.routes.js` : เส้นทางดึงข้อมูลออกรายงานทางการ

ตัวประมวลผลธุรกิจระบบ ('controllers/')

- `auth.controller.js` : จัดการ Login เปรียบเทียบรหัสผ่าน Bcrypt และสร้าง JWT Token
- `project.controller.js` : บันทึกโครงการ, คำนวณ % ก้าวหน้า (`updateProjectProgress`), ล็อกแผนงาน (Plan Locking) และบันทึกข้อสั่งการพร้อมตรวจ IDOR
- `activity.controller.js` : บันทึกกิจกรรมย่อย งบที่ใช้จริง และส่งคำนวณ % โครงการแม่ใหม่ทันที
- `dashboard.controller.js` : คำนวณสรุปสถิติผู้บริหาร และตรวจจับ โครงการติดธงแดง (Red Flags)
- `master.controller.js` : เพิ่ม/ลบ/แก้ไข ข้อมูลระบบพื้นฐานทั้ง 9 หมวด
- `issue.controller.js` : รับเรื่องแจ้งปัญหา บันทึกข้อความตอบกลับของ Admin
- `report.controller.js` : ประมวลผลตัวเลขและสถิติสำหรับหน้าจอรายงาน

3) หมวดหน้าบ้าน ('frontend/')

ไฟล์โครงสร้างหลักและคอนฟิก:

- `index.html` : หน้าเว็บโครงร่างหลัก มี `<div id="root"></div>` สำหรับเรนเดอร์ React
- `src/main.jsx` : จุด Mount สคริปต์ React เข้ากับหน้า `index.html`
- `src/App.jsx` : จัดการระบบเส้นทางหน้าจอ (Routes) ทั้งหมด หุ้มด้วย `ProtectedRoute`
- `src/index.css` : สไตล์ Global, ฟอนต์ Prompt, สไตล์ Select และ Scrollbar
- `tailwind.config.js` : กำหนดธีมสีหลัก (`primary: #6C3BFF`), สีพื้นหลัง และขนาดฟอนต์
- `vercel.json` : คำสั่ง URL Rewrite ป้องกันหน้า 404 เมื่อ Refresh หน้าเว็บ SPA บน Vercel

เลย์เอาต์และคอมโพเนนต์ส่วนกลาง ('components/', 'layouts/')

- `layouts/AppLayout.jsx` : โครงร่างหลัก รวม Topbar + Sidebar + `<Outlet />`
- `components/Topbar.jsx` : แถบหัวเว็บ แสดงชื่อหน้าภาษาไทยตาม Role, กระดิ่งแจ้งเตือน, กล้องโปรไฟล์ผู้ใช้
- `components/Sidebar.jsx` : แถบเมนูด้านซ้าย แสดงลิงก์นำทางตามสิทธิ์ของผู้ใช้งาน
- `components/CustomSelect.jsx` : Dropdown พิเศษ เรนเดอร์แบบ Portal รองรับตัวเลือกยาวและค้นหาได้
- `components/ExecutiveProjectModal.jsx` : Modal แสดงสรุปโครงการและช่องบันทึกข้อสั่งการของผู้บริหาร
- `components/ProfileModal.jsx` : Modal แก้ไขโปรไฟล์และเปลี่ยนรหัสผ่าน
- `components/ReportIssueModal.jsx` : Modal ส่งแจ้งปัญหาระบบไปยังผู้ดูแลระบบ
- `components/ErrorBoundary.jsx` : ดักจับข้อผิดพลาดของ React ไม่ให้หน้าจอดับ

สเตตและบริการเชื่อมต่อ (`contexts/`, `services/`, `utils/`):

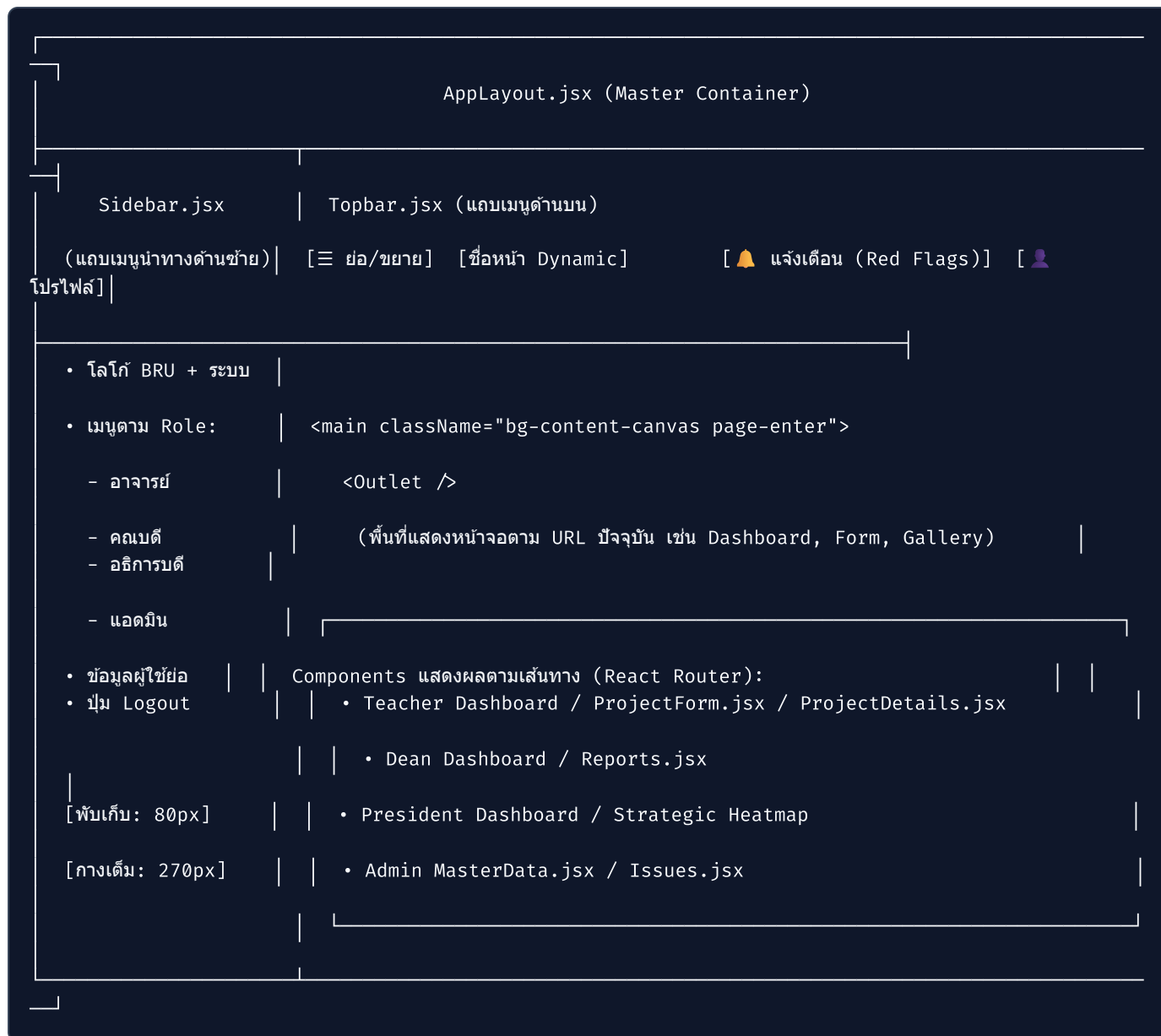
- `contexts/AuthContext.jsx` : เก็บข้อมูล User และ Token ปัจจุบัน พร้อมฟังก์ชัน Login / Logout
- `services/api.js` : Axios Instance กลาง แแนบ Header `Authorization: Bearer <token>` อัตโนมัติ
- `utils/imageUrl.js` : แปลง Path รูปภาพ และบีบอัดรูปภาพก่อนส่งขึ้นเซิร์ฟเวอร์
- `utils/statusHelper.js` : คำนวณสี Badge และข้อความสถานะโครงการ

หน้าจของผู้ใช้งานแยกตาม Role (`pages/`):

- `auth/Login.jsx` : หน้าจอล็อกอินเข้าสู่ระบบ
- `teacher/` (อาจารย์): จัดการแดชบอร์ดงานตนเอง, สร้าง/แก้ไขโครงการ, รายละเอียดโครงการ, บันทึกกิจกรรม, แกลเลอรีรูปภาพ
- `dean/` (คณบดี): แดชบอร์ดสรุปผลงานระดับคณะ, ตารางเปรียบเทียบสาขาวิชา, รายงานคณะ
- `president/` (อธิการบดี): Strategic Dashboard มหาวิทยาลัย, ตรวจสอบโครงการติดธงแดง, รายงานระดับสถาบัน
- `admin/` (ผู้ดูแลระบบ): แดชบอร์ดระบบ, จัดการข้อมูล Master Data 9 หมวด, ปลดล็อกโครงการ, จัดการปัญหา
- `executive/ProjectDetail.jsx` : หน้ารายละเอียดโครงการแบบสรุปสำหรับผู้บริหาร

4.3 โครงสร้างหน้าจหลักและองค์ประกอบ UI (Master Layout: Topbar, Sidebar, Main Canvas)

ระบบใช้สถาปัตยกรรม **Master Layout Pattern** ที่รวมศูนย์โครงสร้างหน้าจไว้ในที่เดียว เพื่อให้ทุกหน้าจามีรูปแบบที่สวยงาม สม่ำเสมอ และปรับเปลี่ยนตามบทบาทผู้ใช้งาน (Role) ได้อย่างสมบูรณ์



1) โครงสร้างแม่แบบหลัก: `frontend/src/layouts/AppLayout.jsx`

- ไฟล์ที่อยู่: `frontend/src/layouts/AppLayout.jsx`
- หน้าที่: เป็นตัวจัดระเบียบโครงสร้างหลัก (Root Shell) ที่ห่อหุ้มทุกหน้าจอในระบบ
- องค์ประกอบและตรรกะภายใน:

1. ตัวจัดการสถานะเมนู (State Management):

- `sidebarOpen` (Boolean): ควบคุมการเปิด-ปิดเมนูบนหน้าจออุปกรณ์พกพา (Mobile Off-canvas Drawer)
- `sidebarCollapsed` (Boolean): ควบคุมการยุบแถบเมนูด้านซ้ายให้เหลือเฉพาะไอคอน (`80px`) หรือขยายเต็ม (`270px`) บนหน้าจอคอมพิวเตอร์
- กำหนด CSS Variable `--sidebar-width` ปรับขนาดความกว้างแบบเรียลไทม์

2. การจัดวางแบบ Flexbox: จัดแบ่งหน้าจอเป็น 2 ฝั่งซ้าย-ขวา โดยเมนูซ้ายมือคงที่ และฝั่งขวามือเป็นพื้นที่เลื่อนดูเนื้อหาแบบอิสระ (`overflow-y-auto`)

3. การรองรับการพิมพ์เอกสาร (Print Media): ใส่คลาส `print:overflow-visible print:h-auto print:block` เพื่อให้เมื่อผู้ใช้กดพิมพ์รายงาน (Ctrl + P) หน้าจอจะตัดแถบเมนูออกและจัดพิมพ์เนื้อหาได้อย่างสวยงามเต็มหน้ากระดาษ

2) แถบเมื่อนำทางด้านซ้าย (Menu Bar / Sidebar): `frontend/src/components/Sidebar.jsx`

- ไฟล์ที่อยู่: `frontend/src/components/Sidebar.jsx`
- หน้าที่: แสดงรายการเมื่อนำทางของระบบ โดยระบบจะคัดกรองลิงก์เมนูให้เห็นเฉพาะสิทธิ์ของผู้ใช้นั้นโดยอัตโนมัติ (Role-based Navigation)
- องค์ประกอบสำคัญภายในแถบเมนู:
 1. Branding Header (ส่วนหัวเมนู):
 - แสดงโลโก้มหาวิทยาลัยราชภัฏบุรีรัมย์ และชื่อระบบ "BRU Strategic Tracking"
 - พื้นหลังไล่เฉดสีม่วงเข้มเป็นเอกลักษณ์ (`linear-gradient(180deg, #2F1481 0%, #1E0A4A 100%)`)
 2. ปุ่มย่อ/ขยายเมนู (Collapse Button):
 - ปุ่มลูกศรยุบเมนูซ่อนข้อความให้เหลือเฉพาะไอคอน 80px ช่วยเพิ่มพื้นที่แสดงผลบนจอเดสก์ท็อป
 3. รายการเมนูที่ปรับเปลี่ยนตามบทบาท (Role-based NavLinks):
 - อาจารย์ (`TEACHER`): แดชบอร์ดงานของฉัน (`/`), รายการโครงการ (`/projects`), รายการกิจกรรม (`/activities`), แกลเลอรีภาพกิจกรรม (`/gallery`)
 - คณบดี (`DEAN`): แดชบอร์ดภาพรวมคณะ (`/`), โครงการ & Red Flags คณะ (`/projects`), รายงานสรุประดับคณะ (`/reports`)
 - อธิการบดี (`PRESIDENT`): แดชบอร์ดภาพรวมสถาบัน (`/`), Strategic Heatmap โครงการ (`/projects`), รายงานสรุปยุทธศาสตร์ (`/reports`)
 - ผู้ดูแลระบบ (`ADMIN`): แดชบอร์ดภาพรวมระบบ (`/`), จัดการข้อมูลระบบ 9 หมวด (`/master-data`) มีเมนูย่อยแบบพับได้ เช่น ผู้ใช้, ยุทธศาสตร์ 4 ระดับ, ปิงปวงประมาณ, คณะ, สาขา), รายงานสรุป (`/reports`), รายงานปัญหาระบบ (`/admin/issues`)
 4. User Profile Footer (ส่วนท้ายแถบเมนู):
 - แสดงชื่อ-นามสกุล, ป้ายกำกับบทบาท (เช่น "คณบดี", "อาจารย์ / ผู้รับผิดชอบ"), คณะ/หน่วยงานสังกัด
 - ปุ่มออกจากระบบ (Logout) แบบคว่ำ

3) แถบเมนูด้านบน (Topbar / Header): `frontend/src/components/Topbar.jsx`

- ไฟล์ที่อยู่: `frontend/src/components/Topbar.jsx`
- หน้าที่: แถบควบคุมด้านบน แสดงชื่อหน้าจปัจจุบัน ระบบแจ้งเตือน และจัดการบัญชีผู้ใช้งาน
- องค์ประกอบสำคัญภายใน Topbar:
 1. ปุ่มเปิด-ปิดเมนู (Hamburger Toggle Button):

- ไอคอน `FiMenu` สำหรับกดเปิดแถบเมนูข้างบนมือถือ หรือย่อ-ขยายบนคอมพิวเตอร์

2. ชื่อหน้าจอบแบบไดนามิก (Dynamic Page Title):

- มีฟังก์ชัน `getPageTitle(pathname, search, role)` คำนวณชื่อหัวข้อตาม URL และบทบาท เช่น:
 - อยู่หน้าแรก (/): หากเป็นอาจารย์จะขึ้น "แดชบอร์ดงานของฉัน", คณบดีจะขึ้น "แดชบอร์ดภาพรวมคณะ", อธิการบดีจะขึ้น "แดชบอร์ดภาพรวมสถาบัน"
 - อยู่หน้าจัดการข้อมูล: จะแสดงชื่อหมวดย่อยตามแท็บที่เปิด เช่น "จัดการประเด็นการพัฒนาท้องถิ่น", "จัดการข้อมูลคณะ"

3. ศูนย์แจ้งเตือนอัจฉริยะ (Notification Center - กระดิ่ง `FiBell`):

- ดึงข้อมูลโครงการที่มีปัญหาหรือโครงการล่าช้า (Red Flags) มาแจ้งเตือน
- มีตัวเลข Badge สีแดงนับจำนวนรายการแจ้งเตือนที่ยังไม่ได้อ่าน
- เมื่อคลิกจะเปิด Dropdown แสดงรายการแจ้งเตือน พร้อมปุ่ม "ทำเครื่องหมายว่าอ่านแล้ว", "เลือกหลายรายการ" และ "ลบการแจ้งเตือน" โดยบันทึกสถานะลง `localStorage` แยกตามรหัสผู้ใช้ (`user.id`)

4. กล่องข้อมูลผู้ใช้และเมนูจัดการบัญชี (User Profile Dropdown):

- แสดงภาพประจำตัว (Avatar) หากยังไม่มีรูปจะแสดงตัวอักษรย่อ พร้อมชื่อผู้ใช้และตำแหน่ง
- เมื่อคลิกรูปโปรไฟล์จะเปิด Dropdown Menu มี 4 ตัวเลือกสำคัญ:
 - ข้อมูลโปรไฟล์: เรียกเปิด `ProfileModal.jsx` เพื่อดูรายละเอียดและอัปโหลดภาพประจำตัวใหม่
 - เปลี่ยนรหัสผ่าน: มีฟอร์มกรอกรหัสผ่านเดิมและรหัสผ่านใหม่ พร้อมการตรวจสอบความปลอดภัย
 - แจ้งปัญหาระบบ: เรียกเปิด `ReportIssueModal.jsx` ให้ผู้ใช้ส่งภาพและข้อความแจ้งปัญหาตรงถึงผู้ดูแลระบบ
 - ออกจากระบบ (Logout): ล้าง Token ใน `localStorage` และนำทางกลับสู่หน้าล็อกอิน

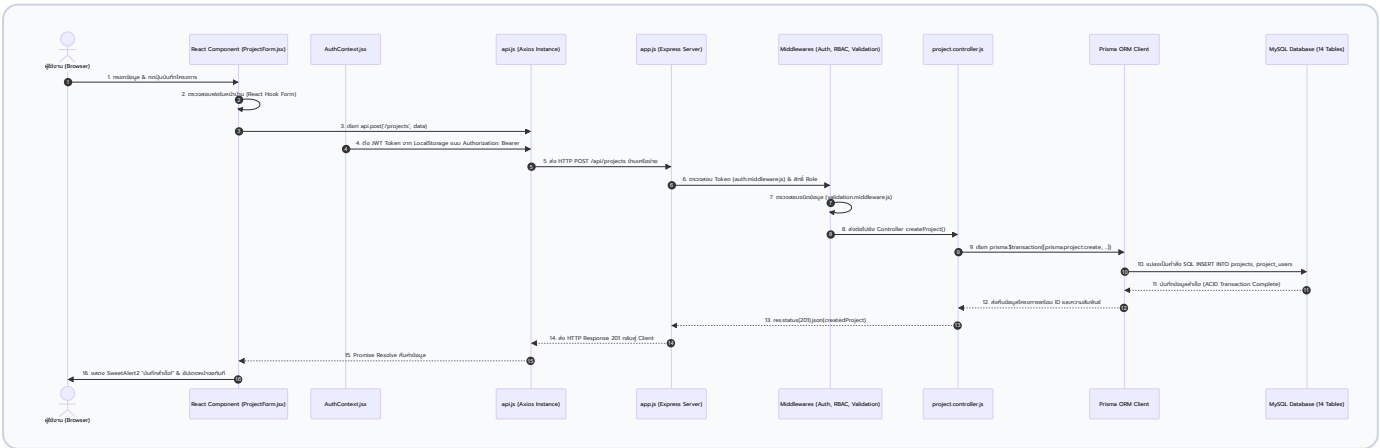
4) พื้นที่แสดงผลเนื้อหาหลัก (Main Content Canvas): ``

- ไฟล์ที่อยู่: เชื่อมโยงใน `frontend/src/layouts/AppLayout.jsx` แสดงผลตามเส้นทางใน `frontend/src/App.jsx`
- หน้าที: เรนเดอร์ Component หน้าจอแต่ละหน้าตาม URL ที่ผู้ใช้เปิดใช้งาน
- คุณสมบัติการออกแบบ:
 - พื้นหลังโทนสีนุ่มนวลสบายตา `bg-content-canvas` (`#F8FAFC`) ช่วยขบเน้น Card ข้อมูลสีขาวให้เด่นชัด
 - ใช้ `key={location.pathname}` ร่วมกับแอนิเมชัน `page-enter` เพื่อให้หน้าจอค่อยๆ ปรากฏอย่างนุ่มนวล (Fade-in) เมื่อเปลี่ยนหน้า
 - รองรับ Responsive Padding (`p-3.5 sm:p-5 md:p-7 lg:p-8`) สวยงามพอดีในทุกขนาดหน้าจอ

5) ตารางสรุปไฟล์องค์ประกอบหน้าเว็บและตำแหน่งจัดเก็บ

องค์ประกอบหน้าเว็บ	ไฟล์ที่จัดเก็บ (File Path)	คอมโพเนนต์หลัก (Component)	จุดเด่นและหน้าที่สำคัญ
โครงสร้างรวมทั้งระบบ	frontend/src/layouts/	AppLayout.jsx	Master Layout ห่อหุ้มโครงสร้างคุม State ย่อ/ขยาย และรองรับโหมดพิมพ์รายงาน
แถบเมนูนำทางด้านซ้าย	frontend/src/components/	Sidebar.jsx	กรองรายการเมนูตาม Role, โลโก้สถาบัน, โลโก้สีม่วง, ยึดเหลือ 80px ได้
แถบหัวเว็บด้านบน	frontend/src/components/	Topbar.jsx	Dynamic Title เปลี่ยนตามหน้า, กระดิ่งเตือน Red Flags, จัดการโปรไฟล์ผู้ใช้
พื้นที่เนื้อหาหน้าเว็บ	frontend/src/layouts/	<Outlet /> (React Router)	แสดง Component ของหน้าจอตตาม URL พร้อมแอนิเมชันสลับหน้า Fade-in
หน้าต่างโปรไฟล์ผู้ใช้	frontend/src/components/	ProfileModal.jsx	หน้าต่างป๊อปอัพแก้ไขข้อมูลส่วนตัวและอัปโหลดรูปภาพประจำตัว
หน้าต่างแจ้งปัญหา	frontend/src/components/	ReportIssueModal.jsx	หน้าต่างป๊อปอัพส่งเรื่องแจ้งปัญหาระบบตรงถึงผู้ดูแลระบบ
หน้าต่างดูโครงการผู้บริหาร	frontend/src/components/	ExecutiveProjectModal.jsx	หน้าต่างป๊อปอัพสรุปใหม่ไลน์โครงการและช่องบันทึกข้อสั่งการ
ตัวเลือกค้นหายุทธศาสตร์	frontend/src/components/	CustomSelect.jsx	Searchable Dropdown เลือกยุทธศาสตร์ 4 ระดับแบบค้นหาข้อความได้

4.4 แผนผังและกลไกการทำงานร่วมกันตั้งแต่หน้าบ้านถึงฐานข้อมูล

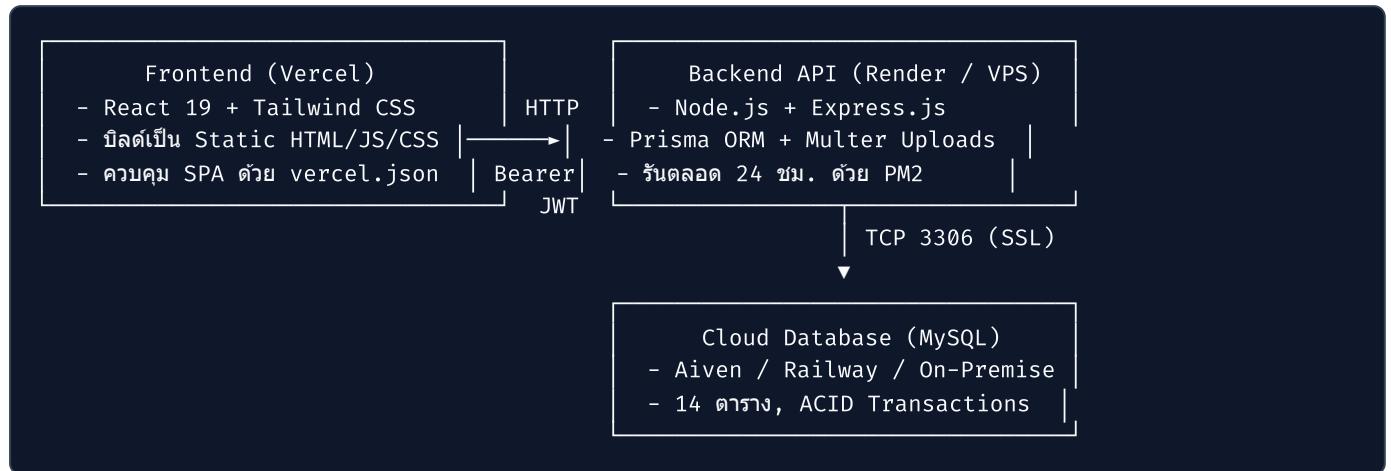


4 กลไกการทำงานร่วมกันที่สำคัญ:

1. **ความปลอดภัยไร้สถานะ (Stateless JWT & RBAC):** หน้าบ้านไม่ต่อฐานข้อมูลตรง ทุกคำขอต้องผ่าน API พร้อมแนบ JWT Token ตรวจสอบสิทธิ์ที่หลังบ้านเสมอ
 2. **ความสมบูรณ์ของข้อมูล (ACID Transactions):** บันทึกหลายตารางพร้อมกันผ่าน `prisma.$transaction` หากจุดใดล้มเหลวจะ Rollback ทันที ข้อมูลไม่ค้าง
 3. **การคำนวณแบบ Real-time:** เมื่อบันทึกกิจกรรมย่อย หลังบ้านจะรวมผลผลิตสะสมและคำนวณ % โครงการแม่ใหม่ทันที ทำให้แดชบอร์ดเป็นปัจจุบันเสมอ
 4. **การควบคุมธรรมาภิบาล (Plan Locking & IDOR):** หน้าบ้านล็อกช่องห้ามแก้ไข และหลังบ้านตรวจสอบซ้ำเพื่อป้องกันการลัดเป้าหมายย้อนหลัง และป้องกันการสั่งการข้ามคณะ
-

5. การ Deploy ระบบ: เครื่องมือและวิธีการทำงาน

ระบบใช้สถาปัตยกรรม **Decoupled Architecture** (แยกหน้าบ้าน-หลังบ้านอิสระจากกัน):



5.1 การ Deploy ฝั่ง Frontend ด้วย Vercel

- **เครื่องมือ:** Vercel
- **การทำงาน:** เมื่อ Push โค้ดขึ้น GitHub ตัว Vercel จะรัน `npm run build` บิลด์โค้ดออกมาเป็นไฟล์ Static HTML/JS/CSS ในโฟลเดอร์ `frontend/dist/`
- **ทำไมต้องมี `vercel.json`:**
 เพราะระบบเป็น Single Page Application (SPA) หากผู้ใช้กด Refresh ที่หน้าย่อยเช่น `/projects/5` ตัวเซิร์ฟเวอร์จะหาไฟล์ไม่เจอและขึ้น 404
 ไฟล์ `vercel.json` จึงสั่ง **URL Rewrite** ให้ส่งทุก Request กลับมาที่ `/index.html` เพื่อให้ React Router ฝั่ง Client จัดการแสดงผลหน้าเดิมได้ถูกต้อง

5.2 การ Deploy ฝั่ง Backend ด้วย Render / Railway หรือ VPS

- **เครื่องมือ:** Render, Railway หรือเซิร์ฟเวอร์ Linux VPS ของมหาวิทยาลัย
- **การทำงาน:** รันด้วย Node.js ร่วมกับ **PM2 Process Manager** (`pm2 start backend/app.js --name "bru-backend"`) เพื่อให้เซิร์ฟเวอร์ทำงานตลอด 24 ชม. และรีสตาร์ทตัวเองอัตโนมัติหากมีปัญหา พร้อมตั้งค่า Nginx ทำ Reverse Proxy รองรับ HTTPS (SSL)

5.3 การเชื่อมต่อฐานข้อมูล (Database Deployment)

- **เครื่องมือ:** Cloud MySQL (Aiven, Railway) หรือเซิร์ฟเวอร์ MySQL ของมหาวิทยาลัย
- **การตั้งค่า Environment Variables (`.env`):**
 - `DATABASE_URL`: Connection string เชื่อมต่อ MySQL
 - `JWT_SECRET`: คีย์ลับสำหรับเข้ารหัส Token
 - `VITE_API_URL` (หน้าบ้าน): URL ขึ้นไปยัง Backend API

6. ชื่อตารางฐานข้อมูล 14 ตารางและความหมาย

ฐานข้อมูลของระบบพัฒนานบน MySQL 8.0 ผ่าน Prisma ORM ประกอบด้วย 14 ตาราง แบ่งออกเป็น 5 กลุ่ม โดเมน:

ตารางที่	ชื่อตารางใน MySQL	Prisma Model	กลุ่มข้อมูล	ความหมายและหน้าที่ในระบบ
1	faculties	Faculty	โครงสร้างองค์กร	เก็บข้อมูลคณะต่าง ๆ ในมหาวิทยาลัย (เป็นขอบเขตข้อมูล Data Isolation ของคณะ)
2	departments	Department	โครงสร้างองค์กร	เก็บข้อมูลภาควิชา/สาขาวิชา ที่สังกัดภายใต้คณะ
3	users	User	โครงสร้างองค์กร	เก็บบัญชีผู้ใช้, รหัสผ่านแฮช Bcrypt, สิทธิ Role (ADMIN , TEACHER , DEAN , PRESIDENT)
4	fiscal_years	FiscalYear	งบประมาณ	เก็บปีงบประมาณ พ.ศ. (เช่น 2567, 2568) และสถานะปีปัจจุบัน (active)
5	budget_sources	BudgetSource	งบประมาณ	เก็บประเภทแหล่งเงินงบประมาณ เช่น งบประมาณแผ่นดิน, งบรายได้ (บ.กศ.), งบกองทุน
6	local_development_issues	LocalIssue	ยุทธศาสตร์ 4 ระดับ	ยุทธศาสตร์ระดับ 1: ประเด็นการพัฒนาท้องถิ่น (มี 4 ด้านหลักตามพันธกิจ มรภ.)
7	strategies	Strategy	ยุทธศาสตร์ 4 ระดับ	ยุทธศาสตร์ระดับ 2: ประเด็นยุทธศาสตร์หลักของมหาวิทยาลัย
8	sub_strategies	SubStrategy	ยุทธศาสตร์ 4 ระดับ	ยุทธศาสตร์ระดับ 3: แผนงานย่อยที่ระบุเป้าหมายเฉพาะเจาะจง
9	indicators	Indicator	ยุทธศาสตร์ 4 ระดับ	ยุทธศาสตร์ระดับ 4: โครงการหลักระดับสถาบัน (Main Project - MP) หรือตัวชี้วัด Macro
10	projects	Project	การดำเนินงานหลัก	ตารางศูนย์กลาง (31 ฟิลด์): เก็บชื่อโครงการ, งบประมาณ, เป้าหมาย, วันที่, % ความก้าวหน้า (progress), ผลผลิตสะสม (completed_count), สถานะล็อกแผน (is_locked), และข้อสั่งการ 3 ระดับ (executive_directive , dean_directive , president_directive)

ตารางที่	ชื่อตารางใน MySQL	Prisma Model	กลุ่มข้อมูล	ความหมายและหน้าที่ในระบบ
11	project_users	ProjectUser	การดำเนินงานหลัก	Junction Table (Many-to-Many): ผูกอาจารย์หลายท่านมาร่วมบริหารโครงการเดียวกัน โดยใช้ Composite Primary Key [project_id, user_id]
12	activities	Activity	การดำเนินงานหลัก	(13 필드): เก็บแผนกิจกรรมย่อย, วันที่จัด, งบประมาณ (budget), งบใช้จริง (actual_budget), ผลผลิตจริง (completed_count), สถานะสำเร็จ (success), และสถานะล็อกกิจกรรม (is_locked)
13	activity_images	ActivityImage	หลักฐานเชิงประจักษ์	เก็บพาราท้อยู่ไฟล์รูปภาพหลักฐานการลงพื้นที่จัดกิจกรรมจริงบนเซิร์ฟเวอร์ (Evidence-based)
14	issue_reports	IssueReport	การดูแลระบบ	เก็บรายการแจ้งปัญหาระบบจากผู้ใช้ส่งถึง Admin พร้อมระดับความสำคัญ และสถานะการแก้ไข

7. คำถามที่คาดว่าจะถามเกี่ยวกับระบบ พร้อมแนวทางการตอบ

Q1: "ทำไมถึงเลือกใช้ Prisma ORM แทนที่จะเขียนคำสั่ง SQL ธรรมดา (Raw SQL)?"

ตอบ:

*"Prisma ORM มีข้อดี 3 เรื่องหลักครับ:

1. **Type Safety:** ช่วยตรวจชนิดข้อมูลและชื่อฟิลด์ตั้งแต่ขั้นตอนเขียนโค้ด ลดข้อผิดพลาด
2. **Security:** มีระบบ Parameterized Queries ป้องกันการโจมตี **SQL Injection ได้ 100%**
3. **Transactional Integrity:** รองรับคำสั่ง `prisma.$transaction` สำหรับการเขียนหลายตารางพร้อมกันตามมาตรฐาน ACID หากคำสั่งใดล้มเหลวระบบจะ Rollback ทันที ข้อมูลจึงไม่ตกค้างเสียหายครับ"

Q2: "ระบบล็อกแผนงาน (Plan Locking) มีขั้นตอนการทำงานอย่างไร และมีประโยชน์อย่างไร?"

ตอบ:

"มีเพื่อ **รักษาธรรมาภิบาลข้อมูล (Data Governance)** ครับ ป้องกันไม่ให้อาจารย์แอบลดเป้าหมายตัวชี้วัดย้อนหลัง เพื่อให้ % สูงขึ้น

เมื่อโครงการเริ่มบันทึกกิจกรรมย่อยในตาราง `activities` แล้ว หน้าบ้านจะปิด Disabled ช่องงบและเป้าหมาย ขึ้นแม่กุญแจ 🔒

และหลังบ้านใน `project.controller.js` จะตรวจสอบเงื่อนไข หากไม่ใช่สิทธิ์ `ADMIN` ระบบจะส่ง HTTP 400 ปฏิเสธการแก้ไขทันทีครับ"

Q3: "สูตรคำนวณร้อยละความก้าวหน้าของโครงการคิดคำนวณจากอะไร?"

ตอบ:

"คิดจาก **ผลผลิตเชิงปริมาณที่ทำได้จริงสะสม** เทียบกับเป้าหมายที่ตั้งไว้ของโครงการ ครับ

โดยหลังบ้านจะใช้คำสั่ง `.reduce()` รวบรวม `completedCount` จากทุกกิจกรรมของโครงการ แล้วนำมารวมด้วย `targetCount` คูณ 100 จัดรูปแบบทศนิยม 2 ตำแหน่ง พร้อมใช้ `Math.min(... , 100.0)` ป้องกันตัวเลขเกิน 100% และคำนวณใหม่แบบ Real-time ทุกครั้งที่มีการบันทึกกิจกรรมครับ"

Q4: "ระบบตรวจจับโครงการติดธงแดง (Red Flags / Critical Bottlenecks) ใช้เกณฑ์อะไรในการคัดกรอง?"

ตอบ:

*"ใช้หลักการบริหารโดยยึดข้อยกเว้น **Management by Exception (MBE)** ครับ โดยใน

`dashboard.controller.js` ตรวจสอบ 2 เงื่อนไข:

1. วันที่ปัจจุบันเลยกำหนดวันสิ้นสุดโครงการแล้วแต่ยังไม่ครบ 100%
 2. เวลาดำเนินงานผ่านไปเกินครึ่งหนึ่ง (50%) แต่งานทำได้จริงไม่ถึง 25%
- หากเข้าเงื่อนไขข้อใดข้อหนึ่ง ระบบจะติดสถานะธงแดงส่งขึ้นกล่อง 'โครงการที่ต้องเร่งรัดติดตาม' บนแดชบอร์ดผู้บริหารทันที เพื่อให้ผู้บริหารกดออกข้อสั่งการเร่งรัดงานได้ตรงจุดครับ**

Q5: "ทำไมคนไม่ดีถึงไม่เห็นข้อมูลของคนอื่น และระบบป้องกันการแอบยิง API ข้ามคนได้อย่างไร?"

ตอบ:

*"ใช้แนวคิด **Multi-tenancy** และป้องกันช่องโหว่ **IDOR** ครับ:

1. ตอนดึงข้อมูลแดชบอร์ด Backend จะนำ `facultyId` จาก JWT Token ของคนดีไปครอบในเงื่อนไข Query เสมอ
2. ในการออกข้อสั่งการ ฟังก์ชัน `updateExecutiveDirective` จะตรวจสอบว่ารหัสคนของคณบดีตรงกับคนะเจ้าของโครงการหรือไม่ หากไม่ตรงจะตอบกลับด้วย **HTTP 403 Forbidden** ทันที ทำให้ไม่สามารถสั่งการข้ามคนะได้ครับ**

Q6: "รูปภาพหลักฐานกิจกรรมจัดเก็บอย่างไร และทำไมไม่เก็บเป็นไบนารีในฐานข้อมูล?"

ตอบ:

"เราจัดเก็บไฟล์ภาพจริงไว้บนโฟลเดอร์ `/uploads` ของเซิร์ฟเวอร์โดยใช้ Library **Multer** และในฐานข้อมูลตาราง `activity_images` จะเก็บเฉพาะ **URL Path สัมพันธ์** เท่านั้น

เพราะหากเก็บไฟล์ภาพเป็นก้อน Binary (BLOB) ลงในฐานข้อมูล จะทำให้ขนาดฐานข้อมูลขยายใหญ่ขึ้นอย่างรวดเร็ว ส่งผลกระทบต่อความเร็วในการค้นหาและ Backup ข้อมูลอย่างมาก และหน้าบ้านยังมีฟังก์ชันบีบอัดภาพก่อนส่งขึ้นเซิร์ฟเวอร์ด้วยครับ"

Q7: "การยืนยันตัวตนด้วย JWT มีขั้นตอนอย่างไร และหาก Token หลุดมีความเสี่ยงไหม?"

ตอบ:

"ระบบใช้ **Stateless Authentication** ด้วย **JWT** ครับ: เมื่อผู้ใช้ล็อกอินสำเร็จ เซิร์ฟเวอร์จะสร้าง Token ที่เข้ารหัสข้อมูลผู้ใช้และกำหนดเวลาหมดอายุ

ฝั่งหน้าบ้านจะเก็บ Token ไว้ใน Storage และมี Axios Interceptor คอยแนบ Header `Authorization: Bearer <token>` ไปกับทุก Request

ความปลอดภัย: Token มีอายุจำกัด และทุกคำขอที่มีการแก้ไขข้อมูลสำคัญจะถูกตรวจสอบสิทธิ์ (Authorize) ซ้ำที่ฝั่ง Server เสมอครับ"

Q8: "ทำไมถึงเก็บข้อมูลงบประมาณเป็น 'Decimal(12, 2)' แทนที่จะเป็น 'Float'?"

ตอบ:

"เพราะชนิดข้อมูล `Float` หรือ `Double` ใช้การคำนวณเลขฐานสองซึ่งจะเกิด **ปัญหาปัดเศษทศนิยมคลาดเคลื่อน (Rounding Errors)**

ในขณะที่ `Decimal(12, 2)` ใน MySQL จะเก็บค่าตัวเลขที่แน่นอน (Exact Value) รองรับจำนวนเงินได้ถึง 9,999,999.99 บาท ทำให้อุดมงบประมาณและการเบิกจ่ายไม่คลาดเคลื่อนแม้แต่สตางค์เดียวครับ"

Q9: "ความสัมพันธ์ Many-to-Many ระหว่างโครงการและอาจารย์ทำงานอย่างไร?"

ตอบ:

"เนื่องจาก 1 โครงการสามารถมีอาจารย์ผู้รับผิดชอบร่วมได้หลายท่าน และอาจารย์ 1 ท่านสามารถร่วมทำได้หลายโครงการ เราจึงสร้างตารางเชื่อมโยง (Junction Table) ชื่อว่า `project_users` ครับ

โดยกำหนด **Composite Primary Key** เป็น `idid([projectId, userId])` ซึ่งช่วยป้องกันการมอบหมายอาจารย์ท่านเดิมซ้ำในโครงการเดียวกัน และเมื่อโครงการถูกลบ ข้อมูลการผูกในตารางนี้จะถูกลบตามอัตโนมัติด้วย

`onDelete: Cascade` ครับ"

Q10: "ทำไมต้องมีไฟล์ `vercel.json` ในทาง Deploy Frontend?"

ตอบ:

"เพราะแอปพลิเคชันของเราเป็น Single Page Application (SPA) ที่จัดการเส้นทางหน้าจด้วย `react-router-dom` ฝั่งไคลเอนต์ครับ

หากไม่มี `vercel.json` เมื่อผู้ใช้เข้าหน้าเว็บผ่าน URL ลึก ๆ เช่น `/projects/5` หรือกด Refresh ตัวเว็บเซิร์ฟเวอร์จะหาไฟล์ `projects/5.html` ไม่เจอและแสดงข้อผิดพลาด 404

คำสั่ง Rewrite ใน `vercel.json` จะสั่งให้ Server ส่งคำขอทุกเส้นทางกลับมาที่ `/index.html` เพื่อให้ React โหลดขึ้นมาอ่าน URL และแสดงหน้าจอที่ถูกต้องต่อไปครับ"

Q11: "ระบบรองรับการแสดงผลบนสมาร์ตโฟนหรือแท็บเล็ต (Responsive) หรือไม่?"

ตอบ:

"รองรับ 100% ครับ เราใช้คลาส Responsive Breakpoints ของ Tailwind CSS เช่น `sm:`, `md:`, `lg:` ในทุกหน้าจอ

ตัวอย่างเช่น เมนู Sidebar ทางซ้าย เมื่อเปิดบนหน้าจอขนาดเล็กจะพับเก็บเป็นแถบเลื่อน Off-canvas และมีปุ่ม Hamburger บน Topbar สำหรับกดเรียกเปิด-ปิด ส่วนตารางข้อมูลจะมีคลาส `overflow-x-auto` เพื่อให้สามารถเลื่อนดูข้อมูลในแนวนอนได้โดยไม่ทำให้หน้าจอล้นครับ"

Q12: "หากในอนาคตมหาวิทยาลัยมีการปรับเปลี่ยนโครงสร้างยุทธศาสตร์ ระบบรองรับได้หรือไม่?"

ตอบ:

"รองรับได้อย่างสมบูรณ์โดยไม่ต้องแก้ไขโค้ดโปรแกรมเลยครับ

เพราะโครงสร้างยุทธศาสตร์ทั้ง 4 ระดับ ปิงบประมาณ และแหล่งเงินทุน ถูกออกแบบให้อยู่ในรูป **Master Data** ทั้งหมด

ผู้ดูแลระบบ (Admin) สามารถล็อกอินเข้าเมนู 'จัดการข้อมูลระบบ' เพื่อเพิ่ม ลบ หรือแก้ไขยุทธศาสตร์ใหม่ได้ทันทีผ่านหน้าเว็บ และฟอร์มสร้างโครงการของอาจารย์จะดึงข้อมูลใหม่มาแสดงผลแบบ Cascading โดยอัตโนมัติครับ"